TL4 Análisis discriminante cuadrático y lineal

Índice

1. Introducción
2. Análisis discriminante lineal (LDA)
3. Análisis discriminante cuadrático (QDA)
4. Ejercicio

Cita

Juan Ciscar, Alfons. "Material docente de la asignatura Aprendizaje Automático (APA)". Grado de Inteligencia Artificial (GIAR), Escuela Técnica Superior de Ingeniería Informática (ETSINF), Dep. de Sistemas Informáticos y Computación (DSIC), Universitat Politècnica de València (UPV), primer semestre del curso 2026/27.

Copyright © 2026 Alfons Juan Ciscar

1 Introducción

Clasificadores generativos: expresan **posteriors** en función de **priors** y **densidades condicionales** de las clases

$$p(y = c \mid \boldsymbol{x}, \boldsymbol{\theta}) = \frac{p(\boldsymbol{x} \mid y = c, \boldsymbol{\theta})\, p(y = c, \boldsymbol{\theta})}{\sum_{c'} p(\boldsymbol{x} \mid y = c', \boldsymbol{\theta})\, p(y = c', \boldsymbol{\theta})} \propto p(\boldsymbol{x} \mid y = c, \boldsymbol{\theta})\, p(y = c, \boldsymbol{\theta})$$

Análisis discriminante lineal (LDA): modela las condicionales de clase con Gaussianas de Σ común

Análisis discriminante cuadrático (QDA): modela las condicionales de clase con Gaussianas independientes

Ajuste: la maximización de la log-verosimilitud conjunta conduce a estimadores fáciles de calcular

- Las priors de las clases se estiman como frecuencias relativas, $\hat{\pi}_c = N_c / N$
- LDA requiere calcular las medias empíricas de las clases y la matriz de covarianzas global empírica
- QDA requiere calcular las medias y matrices de covarianzas empíricas de las clases

Objetivo: aplicar LDA y QDA a tareas de aprendizaje automático; imágenes principalmente

2 Análisis discriminante lineal (LDA)

LDA: $\boldsymbol{\theta}_c = (\boldsymbol{\mu}_c^t, \operatorname{vec}(\boldsymbol{\Sigma}))^t$, media de la clase c y matriz de varianzas común para todas las clases

$$p(\boldsymbol{x} \mid y = c, \boldsymbol{\theta}_c) = \mathcal{N}(\boldsymbol{x}_n \mid \boldsymbol{\mu}_c, \boldsymbol{\Sigma})$$

con

$$\hat{\boldsymbol{\mu}}_c = \frac{1}{N_c} \sum_{n:y_n=c} \boldsymbol{x}_n$$

$$\hat{\boldsymbol{\Sigma}} = \frac{1}{N} \sum_c \sum_{n:y_n=c} (\boldsymbol{x}_n - \hat{\boldsymbol{\mu}}_c)(\boldsymbol{x}_n - \hat{\boldsymbol{\mu}}_c)^t$$

LDA en scikit-learn: `LinearDiscriminantAnalysis`

- `tol=0.0001`: suavizado de varianzas y covarianzas

Copyright © 2026 Alfons Juan Ciscar

MNIST

Lectura y reducción de la dimensión: $K = 400$

In [ ]:
```
import numpy as np; import datasets; from sklearn.decomposition import PCA
ds = datasets.load_dataset("ylecun/mnist").with_format("numpy")
X_train = ds['train'][:]['image'].astype(np.float32).reshape(-1, 28*28) / 255.
y_train = ds['train'][:]['label'].astype(np.uint8)
X_test = ds['test'][:]['image'].astype(np.float32).reshape(-1, 28*28) / 255.
y_test = ds['test'][:]['label'].astype(np.uint8)
K = 400; pca = PCA(n_components=K); X_train = pca.fit_transform(X_train); X_test = pca.transform(X_test)
print(X_train.shape, y_train.shape, X_test.shape, y_test.shape)
```

```
(60000, 400) (60000,) (10000, 400) (10000,)
```

Precisión en test: tras aprender `tol` y ajustar LDA con el valor aprendido

In [ ]:
```
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.model_selection import train_test_split, ShuffleSplit, RandomizedSearchCV
from scipy.stats import loguniform; import time; start = time.time()
clf = LinearDiscriminantAnalysis(); G = {'tol': loguniform(1e-9, 1e-1)}
splitter = ShuffleSplit(n_splits=2, test_size=0.1, random_state=23)
RS = RandomizedSearchCV(clf, G, n_iter=10, scoring='accuracy', n_jobs=-1, cv=splitter, pre_dispatch=8)
acc = RS.fit(X_train, y_train).score(X_test, y_test)
print(f"Precisión: {acc:.1%} con {RS.best_params_['tol']:.4f}")
print('Tiempo (hh:mm:ss):', time.strftime('%H:%M:%S', time.gmtime(time.time() - start)))
```

```
Precisión: 87.3% con 0.0017
Tiempo (hh:mm:ss): 00:00:07
```

3 Análisis discriminante cuadrático (QDA)

QDA: $\boldsymbol{\theta}_c = (\boldsymbol{\mu}_c^t, \operatorname{vec}(\boldsymbol{\Sigma}_c))^t$, media y matriz de varianzas de la clase c

$$p(\boldsymbol{x} \mid y = c, \boldsymbol{\theta}_c) = \mathcal{N}(\boldsymbol{x}_n \mid \boldsymbol{\mu}_c, \boldsymbol{\Sigma}_c)$$

con

$$\hat{\boldsymbol{\mu}}_c = \frac{1}{N_c} \sum_{n:y_n=c} \boldsymbol{x}_n$$

$$\hat{\boldsymbol{\Sigma}}_c = \frac{1}{N_c} \sum_{n:y_n=c} (\boldsymbol{x}_n - \hat{\boldsymbol{\mu}}_c)(\boldsymbol{x}_n - \hat{\boldsymbol{\mu}}_c)^t$$

QDA en scikit-learn: `QuadraticDiscriminantAnalysis`

- `reg_param=0.0`: porcentaje de interpolación con $\mathbf{I}$ para suavizar matrices de varianzas

Copyright © 2026 Alfons Juan Ciscar

MNIST

Lectura y reducción de la dimensión: $K = 400$

In [ ]:
```
import numpy as np; import datasets; from sklearn.decomposition import PCA
ds = datasets.load_dataset("ylecun/mnist").with_format("numpy")
X_train = ds['train'][:]['image'].astype(np.float32).reshape(-1, 28*28) / 255.
y_train = ds['train'][:]['label'].astype(np.uint8)
X_test = ds['test'][:]['image'].astype(np.float32).reshape(-1, 28*28) / 255.
y_test = ds['test'][:]['label'].astype(np.uint8)
K = 400; pca = PCA(n_components=K); X_train = pca.fit_transform(X_train); X_test = pca.transform(X_test)
print(X_train.shape, y_train.shape, X_test.shape, y_test.shape)
```

```
(60000, 400) (60000,) (10000, 400) (10000,)
```

loguniform: distribución que seguiremos para muestrear valores de `reg_param`

In [ ]:
```
from scipy.stats import loguniform; loguniform(1e-4, 1e-1).rvs(5)
```

Out[ ]:
```
array([0.0017287 , 0.00036507, 0.01927581, 0.00011162, 0.006165  ])
```

Precisión en test: tras aprender `reg_param` y ajustar QDA con el valor aprendido

In [ ]:
```
from sklearn.discriminant_analysis import QuadraticDiscriminantAnalysis
from sklearn.model_selection import train_test_split, ShuffleSplit, RandomizedSearchCV
import time; start = time.time()
clf = QuadraticDiscriminantAnalysis(); G = {'reg_param': loguniform(1e-4, 1e-1)}
splitter = ShuffleSplit(n_splits=2, test_size=0.1, random_state=23)
RS = RandomizedSearchCV(clf, G, n_iter=10, scoring='accuracy', n_jobs=-1, cv=splitter, pre_dispatch=32)
acc = RS.fit(X_train, y_train).score(X_test, y_test)
print(f"Precisión: {acc:.1%} con {RS.best_params_['reg_param']:.4f}")
print('Tiempo (hh:mm:ss):', time.strftime('%H:%M:%S', time.gmtime(time.time() - start)))
```

```
Precisión: 95.8% con 0.0274
Tiempo (hh:mm:ss): 00:00:08
```

Copyright © 2026 Alfons Juan Ciscar

4 Ejercicio

Ejercicio: aplica LDA y QDA a Fashion-MNIST con $K = 600$ y a CIFAR-10 con $K = 1000$

Precisión (%) en test: LDA no mejora LR en ninguna tarea; QDA sí en MNIST y CIFAR-10

	MNIST	Fashion-MNIST	CIFAR-10
	$K = 400$	$K = 600$	$K = 1000$
LR	92.6	84.4	41.1
LDA	87.3	81.6	40.0
QDA	95.8	76.6	53.0

Copyright © 2026 Alfons Juan Ciscar

Fashion-MNIST

In [ ]:
```
import numpy as np; import datasets; from sklearn.decomposition import PCA
ds = datasets.load_dataset("zalando-datasets/fashion_mnist").with_format("numpy")
X_train = ds['train'][:]['image'].astype(np.float32).reshape(-1, 28*28) / 255.
y_train = ds['train'][:]['label'].astype(np.uint8)
X_test = ds['test'][:]['image'].astype(np.float32).reshape(-1, 28*28) / 255.
y_test = ds['test'][:]['label'].astype(np.uint8)
K = 600; pca = PCA(n_components=K); X_train = pca.fit_transform(X_train); X_test = pca.transform(X_test)
print(X_train.shape, y_train.shape, X_test.shape, y_test.shape)
```

```
(60000, 600) (60000,) (10000, 600) (10000,)
```

In [ ]:
```
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.model_selection import train_test_split, ShuffleSplit, RandomizedSearchCV
from scipy.stats import loguniform; import time; start = time.time()
clf = LinearDiscriminantAnalysis(); G = {'tol': loguniform(1e-9, 1e-1)}
splitter = ShuffleSplit(n_splits=2, test_size=0.1, random_state=23)
RS = RandomizedSearchCV(clf, G, n_iter=10, scoring='accuracy', n_jobs=-1, cv=splitter, pre_dispatch=8)
acc = RS.fit(X_train, y_train).score(X_test, y_test)
print(f"Precisión: {acc:.1%} con {RS.best_params_['tol']:.4f}")
print('Tiempo (hh:mm:ss):', time.strftime('%H:%M:%S', time.gmtime(time.time() - start)))
```

```
Precisión: 81.6% con 0.0000
Tiempo (hh:mm:ss): 00:00:11
```

In [ ]:
```
from sklearn.discriminant_analysis import QuadraticDiscriminantAnalysis
from sklearn.model_selection import train_test_split, ShuffleSplit, RandomizedSearchCV
from scipy.stats import loguniform; import time; start = time.time()
clf = QuadraticDiscriminantAnalysis(); G = {'reg_param': loguniform(1e-4, 1e-1)}
splitter = ShuffleSplit(n_splits=2, test_size=0.1, random_state=23)
RS = RandomizedSearchCV(clf, G, n_iter=10, scoring='accuracy', n_jobs=-1, cv=splitter, pre_dispatch=32)
acc = RS.fit(X_train, y_train).score(X_test, y_test)
print(f"Precisión: {acc:.1%} con {RS.best_params_['reg_param']:.4f}")
print('Tiempo (hh:mm:ss):', time.strftime('%H:%M:%S', time.gmtime(time.time() - start)))
```

```
Precisión: 76.6% con 0.0411
Tiempo (hh:mm:ss): 00:00:15
```

CIFAR-10

In [ ]:
```
import numpy as np; import datasets; from sklearn.decomposition import PCA
ds = datasets.load_dataset("uoft-cs/cifar10").with_format("numpy")
X_train = ds['train'][:]['img'].astype(np.float32).reshape(-1, 32*32*3) / 255.
y_train = ds['train'][:]['label'].astype(np.uint8)
X_test = ds['test'][:]['img'].astype(np.float32).reshape(-1, 32*32*3) / 255.
y_test = ds['test'][:]['label'].astype(np.uint8)
labels = ("airplane", "automobile", "bird", "cat", "deer", "dog", "frog", "horse", "ship", "truck")
K = 1000; pca = PCA(n_components=K); X_train = pca.fit_transform(X_train); X_test = pca.transform(X_test)
print(X_train.shape, y_train.shape, X_test.shape, y_test.shape)
```

```
(50000, 1000) (50000,) (10000, 1000) (10000,)
```

In [ ]:
```
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.model_selection import train_test_split, ShuffleSplit, RandomizedSearchCV
from scipy.stats import loguniform; import time; start = time.time()
clf = LinearDiscriminantAnalysis(); G = {'tol': loguniform(1e-9, 1e-1)}
splitter = ShuffleSplit(n_splits=2, test_size=0.1, random_state=23)
RS = RandomizedSearchCV(clf, G, n_iter=10, scoring='accuracy', n_jobs=-1, cv=splitter, pre_dispatch=8)
acc = RS.fit(X_train, y_train).score(X_test, y_test)
print(f"Precisión: {acc:.1%} con {RS.best_params_['tol']:.4f}")
print('Tiempo (hh:mm:ss):', time.strftime('%H:%M:%S', time.gmtime(time.time() - start)))
```

```
Precisión: 40.0% con 0.0000
Tiempo (hh:mm:ss): 00:00:16
```

In [ ]:
```
from sklearn.discriminant_analysis import QuadraticDiscriminantAnalysis
from sklearn.model_selection import train_test_split, ShuffleSplit, RandomizedSearchCV
from scipy.stats import loguniform; import time; start = time.time()
clf = QuadraticDiscriminantAnalysis(); G = {'reg_param': loguniform(1e-4, 1e-1)}
splitter = ShuffleSplit(n_splits=2, test_size=0.1, random_state=23)
RS = RandomizedSearchCV(clf, G, n_iter=10, scoring='accuracy', n_jobs=-1, cv=splitter, pre_dispatch=32)
acc = RS.fit(X_train, y_train).score(X_test, y_test)
print(f"Precisión: {acc:.1%} con {RS.best_params_['reg_param']:.4f}")
print('Tiempo (hh:mm:ss):', time.strftime('%H:%M:%S', time.gmtime(time.time() - start)))
```

```
Precisión: 53.0% con 0.0006
Tiempo (hh:mm:ss): 00:00:39
```

Copyright © 2026 Alfons Juan Ciscar